

概念：30秒重建记忆框架

30秒速记：一个问题，一套方法，一条边界。

- 问题：** 对话持续增长，模型窗口有限；只保留摘要，可能丢掉以后才需要的细节。
- 方法：** 把信息分层保存，模型通过工具选择记什么、查什么，程序执行读写。
- 边界：** 外部存储可以很大，但单次推理仍受上下文窗口限制。**存着，不等于此刻看得见。**

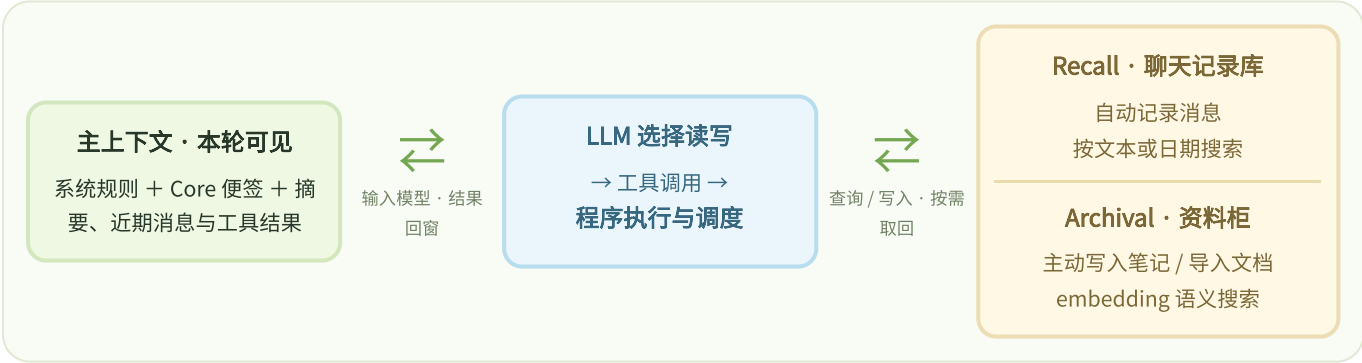
先分清三种记忆

记忆	放什么	谁写入	本轮是否直接可见
Core · 便签	角色、用户偏好、关键事实	模型通过工具编辑	是，全文在系统提示词中
Recall · 聊天记录库	交互消息原文	程序自动记录	窗口外记录需搜索取回
Archival · 资料柜	笔记、导入文档	模型主动写入或文档导入	需要语义检索取回

Core 对应论文的 working context；0.3.1 用 `persona`、`human` 两块文本保存它。它只是主上下文的一部分：模型还会看到固定系统规则、历史摘要、近期消息和工具结果。

从工具反推分工：Core 无需查询工具；Recall 无需让模型逐条写消息；Archival 则分别提供写入与检索。

再看信息怎么流动



读图抓两点： 模型负责选择，程序负责执行；外部记录被取回后，也会消耗上下文 token。旧消息离开窗口时，Recall 中的原文仍然保留，Archival 则不会自动接收每一条被移出的消息。

论文的核心贡献： 把分层存储、自主记忆读写和连续工具调用组合成运行时架构。一条用户消息可以触发多轮“取资料—读结果—再行动”，持续利用跨出窗口的历史。

流程：跟着一次回忆走

用户问：“我上次说想去哪里旅行？”

下面是教学用流程，假设历史原文“下次旅行我想去京都。”只留在历史库中，当前上下文没有这条信息。它不是实测记录；工具调用仅示意关键参数，不是可直接复制的完整请求。

- 1 发现缺口。模型看到问题，但当前窗口中没有答案，需要查询过去的消息。
- 2 发起检索。模型生成工具调用：`conversation_search(query="旅行", request_heartbeat=true)`。
- 3 执行并回填。程序按文本匹配搜索 Recall，把含“旅行”的记录放回消息窗口；heartbeat 请求立即继续一轮推理。
- 4 阅读证据。下一轮模型读到“下次旅行我想去京都。”，提取目的地“京都”。若信息仍不足，可以调整查询或继续取结果。
- 5 发送回答。模型调用 `send_message` 展示“你提过想去京都”。没有预警、错误或 heartbeat 等续跑事件时，程序等待下一次输入。

关键连接： 工具结果先进入上下文，下一轮模型才读到它。`request_heartbeat=true` 请求这次续跑；`send_message` 负责显示答案，本身不强制终止循环。

8个默认工具速查

以下来自 0.3.1 默认聊天与文档预设；六个记忆工具都带有 heartbeat 控制参数。

要做什么	工具
给用户回答；暂停定时唤醒	<code>send_message</code> ； <code>pause_heartbeats</code> （最长360分钟）
修改 Core	<code>core_memory_append</code> ； <code>core_memory_replace</code>
搜索 Recall	<code>conversation_search</code> （文本）； <code>conversation_search_date</code> （日期）
写入／检索 Archival	<code>archival_memory_insert</code> ； <code>archival_memory_search</code> （语义）

窗口快满：分开记住两个动作

- ① 75%：预警，提醒保存要点

0.3.1 在调用返回的 `usage.total_tokens` 超过窗口 75%时告警。模型可选择更新 Core 或写入 Archival；告警不会自动执行存档。
- ② 上下文溢出：摘要，然后重试

程序识别溢出错误后，另调模型把一段旧消息替换为摘要，再重试。Core 与近期消息保留，Recall 中的原文仍在。

论文中的70%预警、满载刷新、约50%驱逐是示例。所查版本的 **75%是预警阈值，不是摘要压缩率**；主动存档与程序生成摘要也不是同一个动作。

判断：证据、选型与自测

实验证明了什么？先记住DMR

任务： 深度记忆检索（DMR）要求回答先前聊天中的具体细节。MSC 每组含五个会话，作者再构造第六个会话中的单个问答；问答由另一 LLM 生成，Accuracy 由 LLM judge 对照标准答案判定。

比较条件： 下表均为同底模比较。基线读取有损历史摘要，MemGPT 可以搜索完整历史。

底层模型	摘要基线	加入 MemGPT
GPT-3.5 Turbo	38.7%	66.9%
GPT-4	32.1%	92.5%
GPT-4 Turbo	35.3%	93.4%

读数边界： 这支持“保留原文并按需找回”在 DMR 中的价值。两组可访问的信息量与访问方式不同，不能仅凭该表拆出各组件贡献，也不能把92.5%推广到其他任务。论文中的 GPT-4／Turbo 分别指 `gpt-4-0613` ／ `gpt-4-1106-preview`。

我的选型判断：有长期信息需求，还要付得起调用成本

- **适合尝试：** 长期个性化助手、项目助理、需反复查证的资料分析。前提是重要事实经常跨出窗口，任务允许额外检索与推理。
- **重点验证：** 是否漏存或写错事实、旧偏好是否更新、查询是否命中、是否过早停止。论文文档问答仅采样50题，也观察到模型常提前停止翻页。
- **验收方法：** 用自己的任务，对齐数据范围与调用预算；分别记录回答正确率、检索次数、token和延迟。记忆做大，不能自动解决推理错误。

五个易混淆点

- **Core改在哪里？** 系统提示词中的工作记忆文本。
- **谁自动保存对话？** 程序写 Recall；Archival 保存主动写入或导入的知识。
- **heartbeat连接哪两步？** 工具结果回窗后，请求下一轮模型推理。
- **75%会触发摘要吗？** 它触发预警；摘要走另一个溢出处理路径。
- **92.5%适用于哪里？** 论文中 GPT-4+MemGPT 的 DMR 评估。

来源与边界： Packer 等，MemGPT，arXiv v2，方法、实验与DMR表；原文 CC BY 4.0。中文翻译、报告与图由 Clearer Notes 整理。

代码为官方0.3.1固定提交的静态核对，涵盖 `agent.py`、`memory.py`、`constants.py`、`main.py`、`functions/`。历史分页接口存在兼容性瑕疵；本报告未运行模型 API，未独立复现实验。